

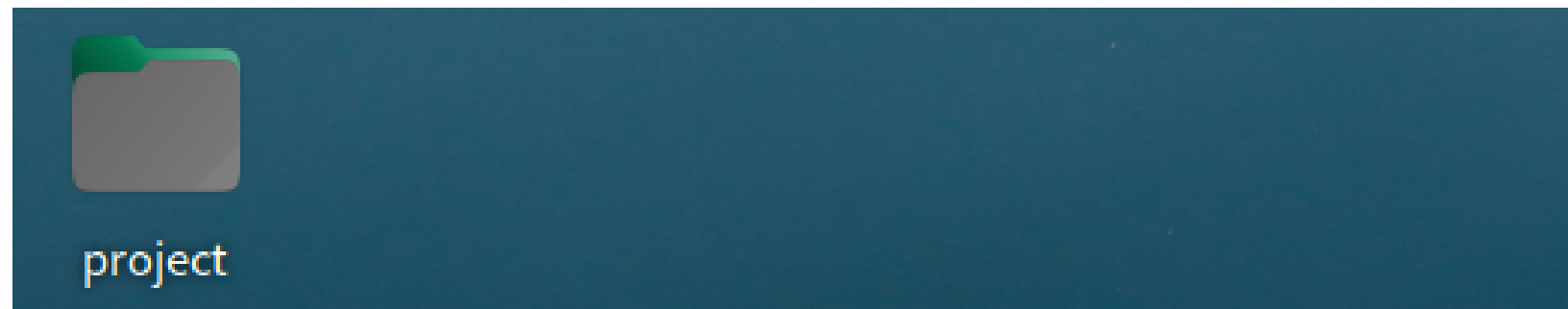


CRUD

CREATE READ UPDATE DELETE
NODE.JS DAN REACT.JS

PERSIAPAN AWAL

Buatlah sebuah direktori yang nantinya digunakan untuk menampung proyek.



Jalankan proyek tersebut dengan menggunakan aplikasi vscode.

```
hk@hk-Linux: ~/Desktop/project
hk@hk-Linux: ~/Desktop/api-users x hk@hk-Linux: ~/Desktop/project x v
hk@hk-Linux:~/Desktop$ cd project/
hk@hk-Linux:~/Desktop/project$ code .
```

Buatlah sebuah folder dengan nama backend.

```
✓ PROJECT  
  > backend
```

Masuk kedalam folder backend dan siapkan sebuah package.json

```
● hk@hk-Linux:~/Desktop/project$ cd backend  
● hk@hk-Linux:~/Desktop/project/backend$ npm init -y
```

Install express, mysql, cors, dan nodemon

```
● hk@hk-Linux:~/Desktop/project/backend$ npm install express mysql cors nodemon
```

Tambahkan script "start":"nodemon server.js" kedalam file package.json

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "nodemon server.js"  
},
```

Tambahkan script "type":"module" kedalam file package.json

```
{  
  "name": "backend",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "type": "module",  
}
```

Install dotenv dengan perintah dibawah

```
● hk@hk-Linux:~/Desktop/project/backend$ npm install dotenv
```

Install prisma pada devDependencies dengan perintah dibawah.

```
● hk@hk-Linux:~/Desktop/project/backend$ npm install -D prisma
```

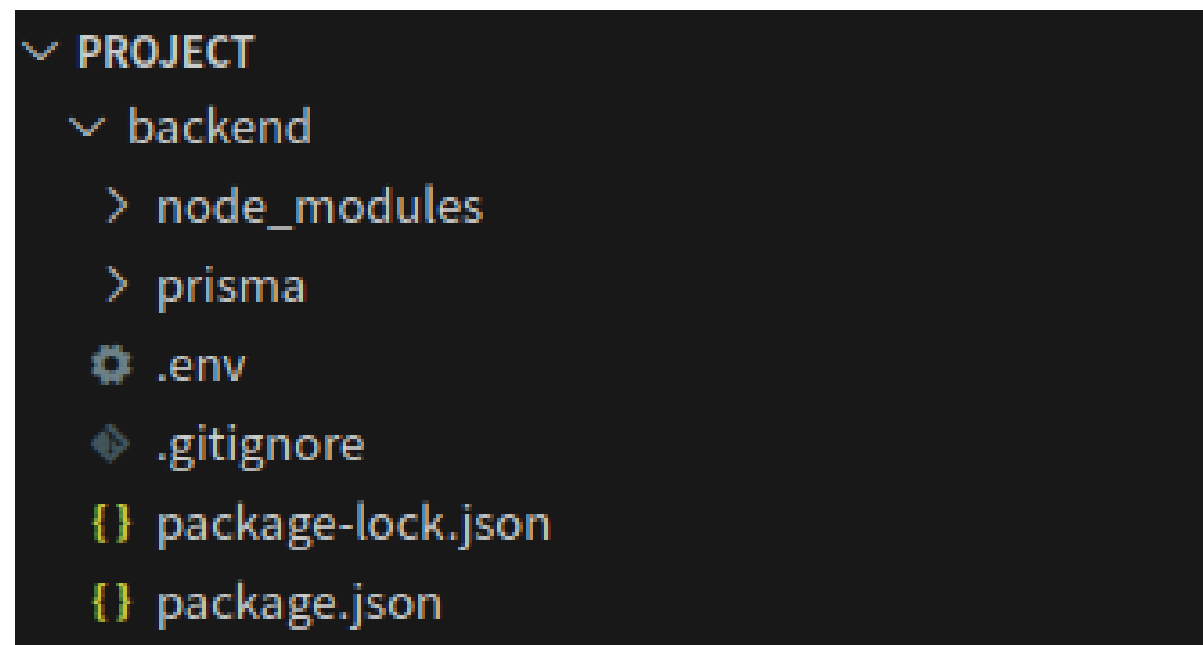
Install prisma client dengan perintah dibawah.

```
● hk@hk-Linux:~/Desktop/project/backend$ npm install @prisma/client
```

inisialisasi prisma dengan perintah dibawah.

```
● hk@hk-Linux:~/Desktop/project/backend$ npx prisma init
```

Maka kita dapat struktur direktori dan package.json sebagai berikut.



```
1 {
2   "name": "backend",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "type": "module",
7   "scripts": {
8     "test": "echo \\\"Error: no test specified\\\" && exit 1",
9     "start": "nodemon server.js"
10  },
11  "keywords": [],
12  "author": "",
13  "license": "ISC",
14  "dependencies": {
15    "@prisma/client": "^6.14.0",
16    "cors": "^2.8.5",
17    "dotenv": "^17.2.1",
18    "express": "^5.1.0",
19    "mysql": "^2.18.1",
20    "nodemon": "^3.1.10"
21  },
22  "devDependencies": {
23    "prisma": "^6.14.0"
24  }
25 }
```



BACKEND

Hapus output yang terletak pada schema.prisma dan ubah postgresql dengan mysql.

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "mysql"  
  url      = env("DATABASE_URL")  
}
```

konfigurasi database pada file .env dan buat database dengan nama siswa

```
DATABASE_URL="mysql://root:Root*123@localhost:3306/siswa"
```

Buatlah sebuah tabel dengan cara membuat model didalam schema.prisma

```
model datasiswa{  
  id Int @id @default(autoincrement())  
  nama String  
  umur Int  
}
```

Buatlah database dan tabel yang telah disiapkan dengan menggunakan migration dengan perintah berikut.

```
● hk@hk-Linux:~/Desktop/project/backend$ npx prisma migrate dev
```

Periksalah tabel yang telah dibuat didalam database.

```
mysql> show databases;
+-----+
| Database |
+-----+
| db_perpustakaan |
| db_review_movie |
| db_univ |
| information_schema |
| mysql |
| performance_schema |
| siswa |
| sys |
| users_db |
+-----+
9 rows in set (0.22 sec)
```



```
mysql> show tables;
+-----+
| Tables_in_siswa |
+-----+
| _prisma_migrations |
| datasiswa |
+-----+
2 rows in set (0.01 sec)
```

Buatlah sebuah file index.js didalam folder backend, kemudian import express, dotenv, dan cors seperti gambar dibawah.

```
import express from "express"
import cors from "cors"
import dotenv from "dotenv"

dotenv.config()

const app=express()

const PORT=8000
app.listen(PORT,()=>{
  console.log("server berjalan");
})
```

Jalankan server dengan perintah dibawah.

```
hk@hk-Linux:~/Desktop/project/backend$ npm start
```

```
> backend@1.0.0 start
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js index.js`
[dotenv@17.2.1] injecting env (0) from .env -- tip: 🔒 encrypt with Dotenvx: https://dotenvx.com
server berjalan
```

Tambahkan middleware pada index.js dengan perintah dibawah.

```
app.use(cors())  
app.use(express.json())
```



```
import express from "express"  
import cors from "cors"  
import dotenv from "dotenv"  
  
dotenv.config()  
  
const app=express()  
|  
app.use(cors())  
app.use(express.json())  
  
const PORT=8000  
app.listen(PORT,()=>{  
|   console.log("server berjalan");  
})
```

Buatlah folder controller dan routes didalam backend.

```
✓ backend  
  > controller  
  > node_modules
```

Buatlah file siswaController.js didalam folder controller dan siswaRoute.js didalam folder Routes.

```
✓ controller
  JS siswaController.js
  > node_modules
  > prisma
  ✓ routes
    JS siswaRoute.js
```

Buatlah beberapa method untuk melihat data, mengupdate data, dan menghapus yang nantinya akan di export data pada file siswaController.js

```
✓ export const getSiswa=async(req, res)=>{
  }

✓ export const getSiswaById=async(req, res)=>{
  }

✓ export const createSiswa=async(req, res)=>{
  }
```

```
✓ export const updateSiswa=async(req, res)=>{
  }
  |
✓ export const deleteSiswa=async(req, res)=>{
  }
```

Import express pada file siswaRouter.js sehingga memungkinkan untuk menggunakan express router dengan code dibawah.

```
import express from "express"
const router = express.Router()

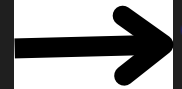
export default router
```

import method yang telah dibuat pada file siswaController.js kedalam file siswaRouter.js menggunakan code dibawah.

```
import express from "express"
import {getSiswa, getSiswaById, createSiswa, updateSiswa, deleteSiswa} from "../controller/siswaController.js"
const router = express.Router()
```

Buatlah route dan endpoint pada file siswaRoute.js sesuai dengan method yang telah di import.

```
router.get('/siswa',getSiswa)
router.get('/siswa/:id',getSiswaById)
router.post('/siswa',createSiswa)
router.put('/siswa/:id',updateSiswa)
router.delete('/siswa/:id',deleteSiswa)
```



```
import express from "express"
import {getSiswa,getSiswaById,createSiswa,updateSiswa,deleteSiswa} from "../controller/siswaController.js"
const router =express.Router()

router.get('/siswa',getSiswa)
router.get('/siswa/:id',getSiswaById)
router.post('/siswa',createSiswa)
router.put('/siswa/:id',updateSiswa)
router.delete('/siswa/:id',deleteSiswa)

export default router
```

import siswaRoute kedalam index.js (entry point) menggunakan code dibawah.

```
import express from "express"
import cors from "cors"
import dotenv from "dotenv"
import siswaRoute from "../routes/siswaRoute.js"
```

Tambahkan sebuah middleware siswaRoute kedalam index.js menggunakan code dibawah.

```
app.use(cors())  
app.use(express.json())  
app.use(siswaRoute)
```



```
import express from "express"  
import cors from "cors"  
import dotenv from "dotenv"  
import siswaRoute from "../routes/siswaRoute.js"  
  
dotenv.config()  
  
const app=express()  
  
app.use(cors())  
app.use(express.json())  
app.use(siswaRoute)  
  
const PORT=8000  
app.listen(PORT,()=>{  
  console.log("server berjalan");  
})
```

Import prisma client kedalam file siswaController.js menggunakan code dibawah.

```
import {PrismaClient} from "@prisma/client"
```

Buatlah sebuah variabel baru dengan nama prisma yang dapat mempermudah penggunaan prisma client.

```
import {PrismaClient} from "@prisma/client"
const prisma = new PrismaClient
```

Tambahkan async dan try-catch pada setiap method yang telah dibuat.

```
export const getSiswa=async(get, res)=>{
  try {

  } catch (error) {

  }
}
```

Buatlah sebuah variabel yang akan digunakan untuk menampilkan data pada method getSiswa.

```
try {
  const reponse = await prisma.siswa.findMany()
  res.status(300).json(reponse)
} catch (error) {
```

Tambahkan code dibawah untuk menampilkan pesan jika terjadi error pada method.

```
} catch (error) {  
  res.status(400).json({message:error.message})  
}
```



```
export const getSiswa=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findMany()  
    res.status(300).json(reponse)  
  } catch (error) {  
    res.status(400).json({message:error.message})  
  }  
}
```

Lakukan pengujian menggunakan prisma studio dan postman/thunderclient.

The screenshot shows a REST client interface with the following details:

- Request:** GET `http://localhost:8000/siswa`
- Response Status:** 300 Multiple Choices
- Response Size:** 35 Bytes
- Response Time:** 115 ms
- Response Body:** A JSON array containing one object:

```
[  
  {  
    "id": 1,  
    "nama": "haris",  
    "umut": 30  
  }  
]
```

Duplikat code yang terdapat pada method getSiswa kemudian paste kedalam method getSiswaById.

```
export const getSiswaById=async(req, res)=>{
  try {
    const reponse = await prisma.siswa.findMany()
    res.status(300).json(reponse)
  } catch (error) {
    res.status(400).json({message:error.message})
  }
}
```

Ubahlah fungsi findMany menjadi findUnique() lalu tambahkan opsi where berdasarkan id yang diambil dari parameter yang dimasukkan.

```
try {
  const reponse = await prisma.siswa.findUnique({
    where:{
      id : req.params.id
    }
  })
  res.status(300).json(reponse)
```

Tambahkan Number/parseInt pada code id sehingga mengubah inputan dari string menjadi number

```
where:{  
  id : Number(req.params.id)  
}
```



```
export const getSiswaById=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findUnique({  
      where:{  
        id : Number(req.params.id)  
      }  
    })  
    res.status(300).json(reponse)  
  } catch (error) {  
    res.status(401).json({message:error.message})  
  }  
}
```

Lakukan pengujian terhadap method yang telah dibuat.

The screenshot shows a REST client interface. On the left, the 'Tests' tab is active, showing a 'Scripting' test. The main area displays a GET request to 'http://localhost:8000/siswa/1' with a 'Send' button. The right panel shows the response details: Status 300 Multiple Choices, Size 33 Bytes, and Time 223 ms. The response body is a JSON object: { "id": 1, "nama": "haris", "umut": 30 }.

select	action	value

Duplikat kembali code yang terdapat pada method getSiswa kemudian paste kedalam method createSiswa.

```
export const createSiswa=async(req, res)=>{
  try {
    const reponse = await prisma.siswa.findMany()
    res.status(300).json(reponse)
  } catch (error) {
    res.status(400).json({message:error.message})
  }
}
```

tambahkan request.body kedalam method createSiswa.

```
export const createSiswa=async(req, res)=>{
  const {nama,umur}=req.body
  try {
    const reponse = await prisma.siswa.findMany()
    res.status(300).json(reponse)
  } catch (error) {
    res.status(400).json({message:error.message})
  }
}
```



Ubahlah fungsi findMany() menjadi create() lalu masukkan opsi berisi data yang akan dikirimkan kedalam database.

```
try {
  const reponse = await prisma.siswa.create({
    data: {
      nama: nama,
      umur: umur
    }
  })
  res.status(300).json(reponse)
```



```
export const createSiswa=async(req, res)=>{
  const {nama,umur}=req.body
  try {
    const reponse = await prisma.siswa.create({
      data: {
        nama:nama,
        umur:umur
      }
    })
    res.status(300).json(reponse)
  } catch (error) {
    res.status(402).json({message:error.message})
  }
}
```

Lakukan pengujian terhadap method createSiswa.

POST http://localhost:8000/siswa/ Send

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "nama": "Haura",
3   "umur": 3
4 }
```

Status: 300 Multiple Choices Size: 32 Bytes Time: 109 ms

Response Headers⁷ Cookies Results Docs

```
1 {
2   "id": 2,
3   "nama": "Haura",
4   "umur": 3
5 }
```

Duplikat code yang terdapat pada method createSiswa kemudian paste kedalam method updateSiswa.

```
export const updateSiswa=async(req, res)=>{
  const {nama,umur}=req.body
  try {
    const reponse = await prisma.siswa.create({
      data: {
        nama:nama,
        umut:umur
      }
    })
    res.status(300).json(reponse)
  } catch (error) {
    res.status(402).json({message:error.message})
  }
}
```

Ubahlah fungsi create() menjadi update() lalu tambahkan opsi where berdasarkan id yang diambil dari parameter yang dimasukkan.

```
const reponse = await prisma.siswa.update({
  where:{
    id : Number(req.params.id)
  },
  data: {
    nama:nama,
    umut:umur
  }
})
```

Lakukan pengujian pada method updateSiswa.

The screenshot shows a REST client interface. The method is PUT, and the URL is http://localhost:8000/siswa/1. The request body is a JSON object: {"nama": "Robi", "umur": 5}. The response status is 300 Multiple Choices, with a size of 31 Bytes and a time of 12 ms. The response body is a JSON object: {"id": 1, "nama": "Robi", "umut": 5}.

Query	Headers ²	Auth	Body ¹	Tests	Pre Run
JSON Content					
1 { 2 "nama": "Robi", 3 "umur": 5 4 }					

Status: 300 Multiple Choices	Size: 31 Bytes	Time: 12 ms
Response		
1 { 2 "id": 1, 3 "nama": "Robi", 4 "umut": 5 5 }		

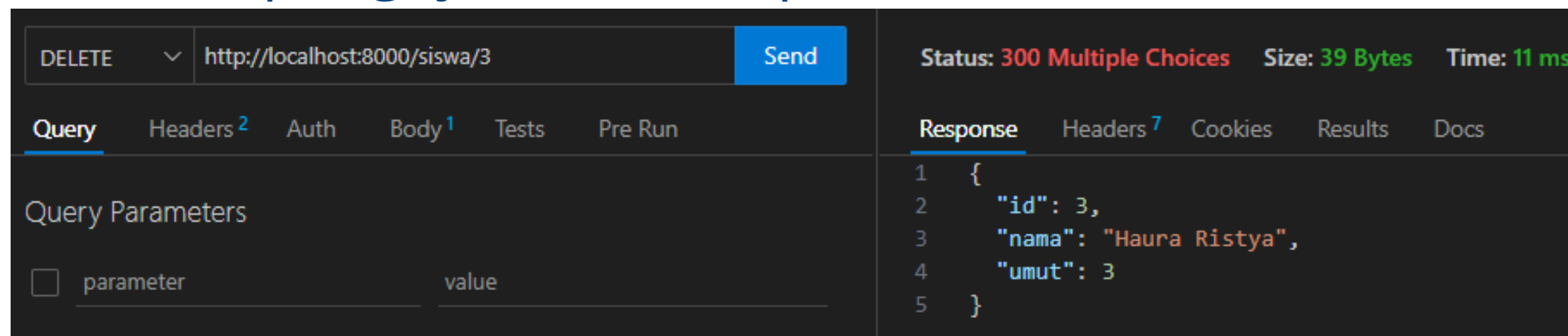
Duplikat code yang terdapat pada method updateSiswa kemudian paste kedalam method deleteSiswa.

```
export const deleteSiswa=async(req, res)=>{
  const {nama,umur}=req.body
  try {
    const reponse = await prisma.siswa.update({
      where:{
        id : Number(req.params.id)
      },
      data: {
        nama:nama,
        umut:umur
      }
    })
    res.status(300).json(reponse)
  } catch (error) {
    res.status(403).json({message:error.message})
  }
}
```

Ubahlah fungsi update() menjadi delete() serta hapus bagian request.body dan opsi where.

```
export const deleteSiswa=async(req, res)=>{
  try {
    const reponse = await prisma.siswa.delete({
      where:{
        id : Number(req.params.id)
      }
    })
    res.status(300).json(reponse)
  } catch (error) {
    res.status(403).json({message:error.message})
  }
}
```

Lakukan pengujian terhadap method deleteSiswa.



The screenshot shows a REST client interface. The top bar displays the method 'DELETE' and the URL 'http://localhost:8000/siswa/3'. The 'Send' button is visible. Below the URL bar, the 'Query' tab is selected, showing 'Query Parameters' with a table for parameters. The 'Response' tab is also visible, showing the JSON response.

parameter	value
☐	

Response: 300 Multiple Choices, Size: 39 Bytes, Time: 11 ms

```
{
  "id": 3,
  "nama": "Haura Ristya",
  "umut": 3
}
```

The image features a white background with a thin dark blue rectangular border. In each of the four corners, there are overlapping geometric shapes: a dark blue triangle and a yellow parallelogram. The word "FRONTEND" is centered in a bold, dark blue, sans-serif font.

FRONTEND

Buka terminal/command prompt baru, kemudian keluar dari folder backend menggunakan perintah `cd ..`. lalu install react beserta direktorinya menggunakan code berikut.

```
D:\hk\project\backend>cd ..  
  
D:\hk\project>npx create-react-app frontend
```

Perhatikan folder/direktori yang ada, jika penginstallan berhasil maka akan tampil seperti berikut.

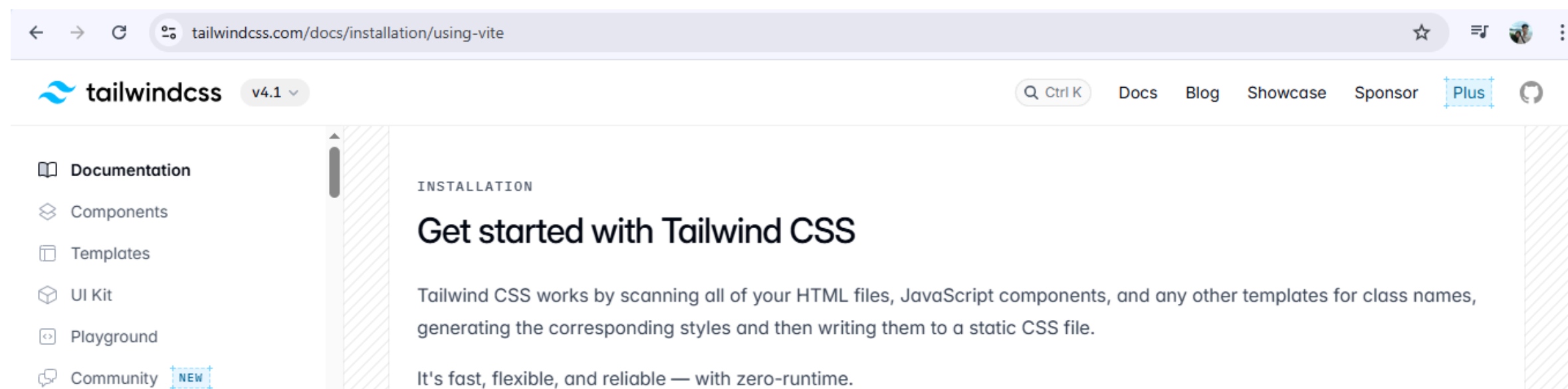
```
▼ PROJECT  
  > backend  
  ▼ frontend  
    > node_modules  
    > public  
    > src  
    .gitignore  
    {} package-lock.json  
    {} package.json  
    ⓘ README.md
```

Masuk kedalam folder frontend, kemudian install axios (karena akan berhubungan dengan api) dan react router dom (karena akan menggunakan router) dan install swr (karena akan menggunakan realtime data fetching).

```
D:\hk\project>cd frontend
```

```
D:\hk\project\frontend>npm install axios react-router-dom swr
```

Lakukan instalasi dengan cara mengikuti guide yang terdapat pada [website resmi tailwind](https://tailwindcss.com/docs/installation/using-vite).



Hapus import logo from './logo.svg' dan ./App.css yang terletak pada file App.js untuk menghilangkan warning.

```
import logo from './logo.svg';  
import './App.css';
```

Hapus file yang tidak diperlukan (App.css, App.test.js, logo.svg, reportWebVitals.js, setupTests.js)

```
# App.css  
JS App.js  
JS App.test.js  
# index.css  
JS index.js  
📁 logo.svg  
JS reportWebVitals.js  
JS setupTests.js
```



```
JS App.js  
# index.css  
JS index.js
```

Hapus reportWebVitals dan function reportWebVitals yang terdapat pada file index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App.js';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Buatlah sebuah folder baru bernama components didalam folder src kemudian buat sebuah file dengan nama daftasSiswa.jsx

```
▼ src
  ▼ components
    🛠 daftasSiswa.jsx
  JS App.js
  # index.css
  JS index.js
```

Generate komponen react dengan menggunakan perintah rafce (jika sudah menginstall plugin) atau ketik code seperti berikut.

```
import React from 'react'

const daftarSiswa = () => {
  return (
    <div>daftarSiswa</div>
  )
}

export default daftarSiswa
```

Bukalah file daftarSiswa.jsx kemudian siapkan sebuah tabel untuk menampung data yang terdapat

```
const daftarSiswa = () => {
  return (
    <div>
      <div>daftarSiswa</div>
      <div>
        <table>
          <thead>
            <tr>
              <th>No</th>
              <th>Nama</th>
              <th>Umur</th>
              <th>Aksi</th>
            </tr>
          </thead>
```

Bukalah file App.js kemudian import react touter dom seperti code dibawah

```
import {BrowserRouter, Routes, Route} from "react-router-dom"
```

Hapus bagian <h1> kemudian ganti dengan BrowserRouter, Routes, dan Route.

```
return (  
  <div>  
    <BrowserRouter>  
      <Routes>  
        <Route/>  
      </Routes>  
    </BrowserRouter>  
  </div>  
>;
```

Tambahkan atribut path "/" yang artinya itu home kemudian tambahkan element untuk merender komponen.

```
<BrowserRouter>  
  <Routes>  
    <Route path="/" element={}/>  
  </Routes>  
</BrowserRouter>
```

import daftarSiswa kedalam file App.js kemudian masukkan daftarSiswa kedalam element yang telah disiapkan.

```
import {BrowserRouter, Routes, Route} from "react-router-dom"
import DaftarSiswa from "../components/daftarSiswa.jsx";
```

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<DaftarSiswa/>}/>
  </Routes>
</BrowserRouter>
```



```
import {BrowserRouter, Routes, Route} from "react-router-dom"
import DaftarSiswa from "../components/daftarSiswa.jsx";

function App() {
  return (
    <div>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<DaftarSiswa/>}/>
        </Routes>
      </BrowserRouter>
    </div>
  );
}

export default App;
```

Import link dari react router dom kedalam file daftarSiswa.jsx

```
import React from "react"
import { Link } from "react-router-dom"
```

Tambahkan sebuah tombol diatas tabel dengan perintah <Link> dan pada kolom aksi ditabel.

```
<Link to="/add">Tambah</Link>
```

```
<td>
  <Link to={` /edit/`} className="bg-blue-500">Edit</Link>
  <button className="bg-red-500">Hapus</button>
</td>
```

Import axios yang digunakan untuk fetching data dan Import SWR karena fetching data akan dilakukan secara realtime pada file daftarSiswa.jsx

```
import axios from "axios"
import useSWR from "swr"
```

Buatlah sebuah method untuk fetching data pada file daftarSiswa.jsx dan masukkan endpoint.

```
const fetcher = async()=>{
  const response=await axios.get('http://localhost:8000/siswa')
  return response.data
}
```

Tambahkan code dibawah ini untuk menggunakan SWR dan tambahkan key yang nantinya akan digunakan

```
const {data}=useSWR('siswa',fetcher)
if(!data) return <h1>Loading</h1>
```

Lakukan pengulangan pada tabel untuk mendapatkan data dari database.

```
<tbody>
  {data.map((siswa,index)=>(
  ))}
  <tr>
```

Pindahkan seluruh kode pada row tbody kedalam pengulangan yang telah dibuat.

```
<tbody>
  {data.map((siswa,index)=>(
    <tr>
      <td></td>
      <td></td>
      <td></td>
      <td>
        <Link to={` /edit/`} className="bg-blue-500">Edit</Link>
        <button className="bg-red-500">Hapus</button>
      </td>
    </tr>
  ))}
</tbody>
```

Tambahkan code dibawah ini untuk menggunakan SWR dan tambahkan key yang nantinya akan digunakan

```
const {data}=useSWR('siswa',fetcher)
if(!data) return <h1>Loading</h1>
```

Lakukan pengulangan pada tabel untuk mendapatkan data dari database.

```
<tbody>
  {data.map((siswa,index)=>(
    ))}
  <tr>
```

Pindahkan seluruh kode pada row tbody kedalam pengulangan yang telah dibuat.

```
<tbody>
  {data.map((siswa,index)=>(
    <tr>
      <td></td>
      <td></td>
      <td></td>
      <td>
        <Link to={`/edit/`} className="bg-blue-500">Edit</Link>
        <button className="bg-red-500">Hapus</button>
      </td>
    </tr>
  ))}
</tbody>
```